— AZ-104 // DEPLOY & MANAGE

Automate Deployment with Templates

ARM templates and Bicep — the structure of a deployment template, deployment modes, scopes, linked templates, and how to validate and troubleshoot a deployment before it breaks production.

Why template-based deployment exists

Clicking through the portal creates resources, but it doesn't record *how* you created them or let you recreate the exact same environment consistently. ARM templates and Bicep turn infrastructure into code — version-controlled, peer-reviewed, repeatable across dev, staging, and production with no drift between environments.

ARM template anatomy — the six sections

$SCHEMA

Schema

Required

URL pointing to the JSON schema that describes the template's structure. Tells tools and validators which template version you're using.

CONTENTVERSION

Version

Required

A version string for the template itself (e.g. `1.0.0.0`). Azure doesn't validate it — it's for your own source control tracking.

PARAMETERS

Parameters

Optional

Values supplied at deploy time — VM name, SKU, environment prefix. Makes templates reusable across environments without editing the template.

VARIABLES

Variables

Optional

Computed values derived from parameters or constants, defined once and referenced throughout. Reduces repetition inside the resources section.

RESOURCES

Resources

Required

The actual Azure resources to deploy — each entry has a `type`, `apiVersion`, `name`, `location`, and `properties` block.

OUTPUTS

Outputs

Optional

Values returned after deployment — resource IDs, connection strings, IP addresses — usable by downstream pipeline steps or linked templates.

Only `$schema`, `contentVersion`, and `resources` are required. Parameters, variables, and outputs are optional but strongly recommended for reusable templates.

Eight terms you need cold

ARM Template

A JSON file that declaratively describes Azure resources and their dependencies. Azure Resource Manager (ARM) reads it and deploys resources in the right order.

Tip: ARM is the underlying engine — both JSON templates and Bicep compile down to ARM API calls.

Bicep

A domain-specific language (DSL) that compiles to ARM JSON. Cleaner syntax, type safety, better IDE support — the recommended authoring experience over raw JSON.

Tip: `az bicep build` compiles `.bicep → .json`. The CLI and Portal accept both directly.

Incremental Mode

The default deployment mode. Adds or updates resources defined in the template but leaves resources in the resource group that are NOT in the template untouched.

Tip: Safe for most deployments — existing resources not in the template are preserved.

Complete Mode

Deploys exactly what's in the template and **deletes** any resources in the resource group that are NOT in the template. Full desired-state enforcement.

Tip: Use with caution — anything not in the template gets deleted. Always validate first.

What-If

A pre-deployment analysis that shows exactly which resources will be created, modified, or deleted without actually making any changes.

Tip: Run What-If before every Complete mode deployment — non-negotiable in production.

Linked Template

A template that is called from a parent template by URI — breaks a large deployment into modular, reusable pieces stored in separate files.

Tip: The linked template file must be accessible via a public URI or a SAS URL — it cannot be a local path at deploy time.

Deployment Scope

The level at which the deployment runs: Resource Group, Subscription, Management Group, or Tenant. The schema URL and CLI command change per scope.

Tip: Most deployments target a resource group — subscription-level deployments are used for policy/RBAC assignments.

dependsOn

Explicitly declares that one resource must be deployed before another. ARM uses this plus implicit dependency detection (via `reference()`) to build the deployment order.

Tip: Prefer implicit dependencies via `reference()` over explicit `dependsOn` — they less likely to go stale

MODE	RESOURCES IN TEMPLATE	RESOURCES NOT IN TEMPLATE	WHEN TO USE
Incremental (default)	Created or updated	**Left alone**	Most deployments — safe default
Complete	Created or updated	Deleted	Full desired-state enforcement — run What-If first

EXAM GOTCHA — COMPLETE MODE DELETES

In Complete mode, **any resource in the resource group that is not declared in the template will be deleted** — including NSGs, NICs, public IPs, and diagnostic settings you added manually. Always run `--what-if` before a Complete mode deployment on any existing resource group.

Template deployment decision flow

Walk through this every time you're deploying or iterating on a template-based deployment.

START

You need to deploy Azure resources repeatably

DECIDE

Are you authoring a new template or deploying an existing one?

AUTHORING NEW

Use Bicep — cleaner syntax, type safety, VS Code extension with IntelliSense

EXISTING JSON

Deploy as-is or use `az bicep decompile` to convert to Bicep

DECIDE

What deployment scope does this target?

RESOURCE GROUP

`az deployment group create` — most common

SUBSCRIPTION

`az deployment sub create` — for policy/RBAC assignments

MGMT GROUP

`az deployment mg create`

DECIDE

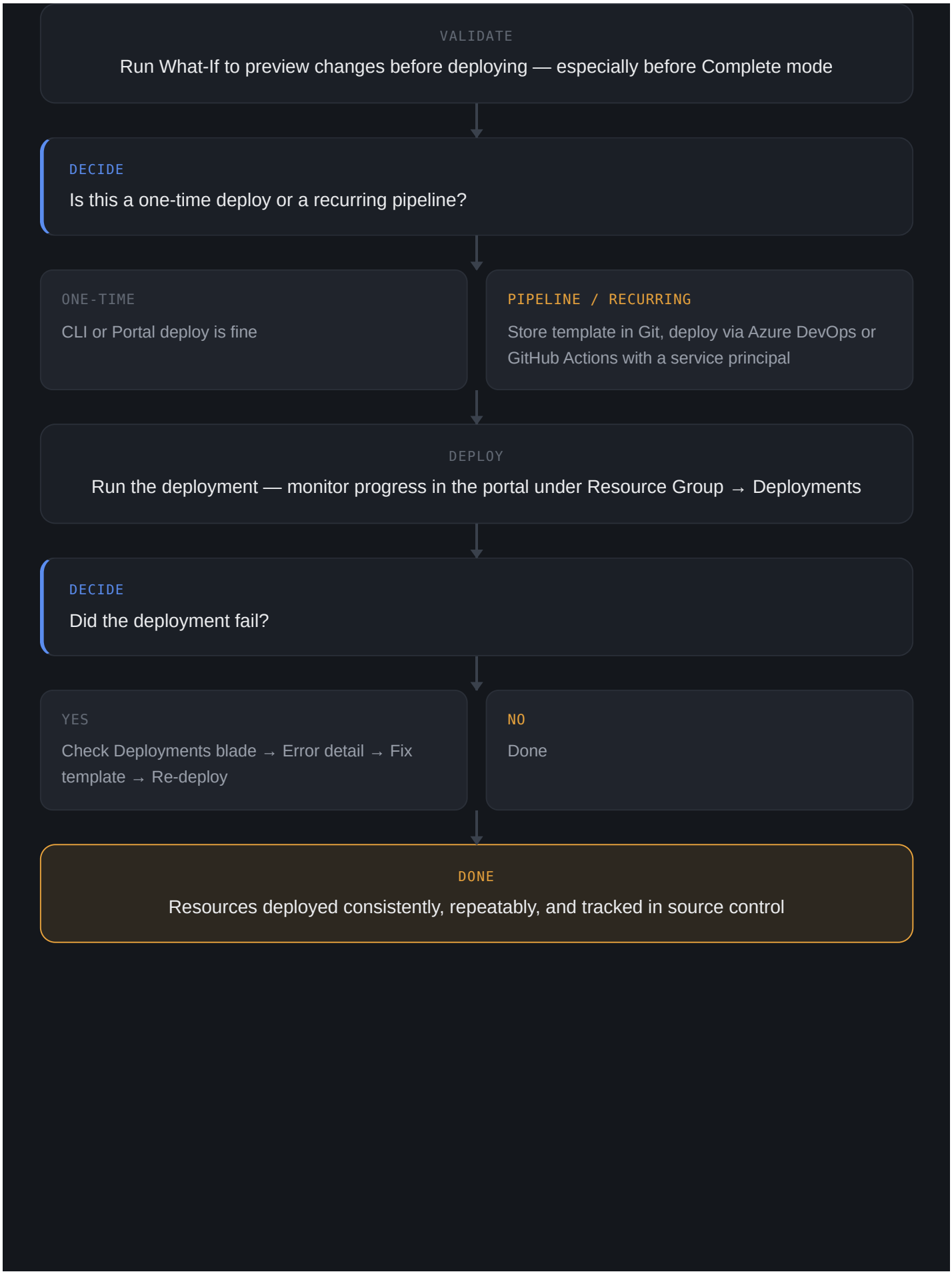
Is this deploying into an existing resource group with other resources?

YES — EXISTING RG

Use Incremental mode (default) to avoid deleting unmanaged resources

NO — CLEAN RG

Complete mode is safe — full desired state with no surprises


VALIDATE
Run What-If to preview changes before deploying — especially before Complete mode
DECIDE
Is this a one-time deploy or a recurring pipeline?
ONE-TIME
CLI or Portal deploy is fine
PIPELINE / RECURRING
Store template in Git, deploy via Azure DevOps or GitHub Actions with a service principal
DEPLOY
Run the deployment — monitor progress in the portal under Resource Group → Deployments
DECIDE
Did the deployment fail?
YES
Check Deployments blade → Error detail → Fix template → Re-deploy
NO
Done
DONE
Resources deployed consistently, repeatably, and tracked in source control

Tools for template deployment

Azure CLI

BEST FOR: scripted deploys, CI/CD pipelines

The primary tool for deploying templates in pipelines. Supports What-If, parameter files, and all four deployment scopes in a single command.

```
# What-If preview
az deployment group what-if \
  --resource-group "rg-prod" \
  --template-file "main.bicep" \
  --parameters @params.json

# Deploy
az deployment group create \
  --resource-group "rg-prod" \
  --template-file "main.bicep" \
  --parameters @params.json \
  --mode Incremental
```

Azure PowerShell

BEST FOR: Windows enterprise pipelines

Full parity with CLI for template deployment — preferred in Windows-centric DevOps environments.

```
New-AzResourceGroupDeployment `
  -ResourceGroupName "rg-prod" `
  -TemplateFile "main.bicep" `
  -TemplateParameterFile
"params.json" `
  -Mode Incremental

# What-If
New-AzResourceGroupDeployment `
  -ResourceGroupName "rg-prod" `
  -TemplateFile "main.bicep" `
  -WhatIf
```

Azure Portal

BEST FOR: one-off deploys, template exploration

The "Deploy a custom template" blade accepts ARM JSON and provides a form for parameter input. The VM wizard can also export its template at the review step.

```
Home → Deploy a custom template
  → Build your own template in editor
  → Load file / paste JSON or Bicep
  → Fill parameters → Review + create
```

VS Code + Bicep Extension

BEST FOR: authoring Bicep templates

The official Bicep VS Code extension provides IntelliSense, type checking, resource snippets, and visualization of the template's dependency graph — the best authoring environment.

```
# Install Bicep CLI
az bicep install

# Compile Bicep to ARM JSON
az bicep build --file main.bicep

# Decompile ARM JSON to Bicep
az bicep decompile --file main.json
```

Azure DevOps / GitHub Actions

BEST FOR: automated CI/CD pipelines

Trigger template deployments on PR merge or push — authenticate with a service principal or OIDC federated credential, no stored secrets needed.

```
# GitHub Actions (azure/arm-deploy
action)
- uses: azure/arm-deploy@v1
  with:
    resourceGroupName: rg-prod
    template: ./main.bicep
    parameters: ./params.json
    deploymentMode: Incremental
```

TOOL	SUPPORTS WHAT-IF?	SUPPORTS BICEP?	PIPELINE-READY?
CLI	**Yes**	**Yes**	**Yes**
PowerShell	**Yes**	**Yes**	**Yes**
Portal	No	**Yes**	No
VS Code Bicep	No (author only)	**Yes**	No
DevOps / Actions	**Yes**	**Yes**	**Yes — purpose-built**

Principles worth internalizing

SCOPE	CLI COMMAND	COMMON USE
Resource Group	`az deployment group create`	VMs, storage, networking — most deployments
Subscription	`az deployment sub create`	RBAC assignments, policy assignments, resource group creation
Management Group	`az deployment mg create`	Org-wide policy and RBAC
Tenant	`az deployment tenant create`	Root management group policy — rarely used directly

ARM VS. BICEP — CHOOSE BICEP

Bicep and ARM JSON are functionally equivalent — Bicep compiles to ARM JSON before deployment. Choose Bicep for new work: **less verbose** (no quotes on property names, no nested strings), **type-safe** (compile-time errors vs. runtime failures), and **better IDE support**. The AZ-104 exam tests both, so know the ARM JSON structure and the Bicep equivalent.

Six mistakes that show up on the exam

Running Complete mode on a shared resource group

Complete mode deletes everything not in the template — including resources added by other teams or pipelines. Always use Incremental in shared environments.

Skipping What-If before a production deploy

What-If is the only way to see deletions before they happen — always run it, especially before Complete mode deployments.

Pointing a linked template at a local file path

Linked templates must be accessible via a public or SAS URI at deploy time — local paths only work from the machine running the deployment, not from ARM's servers.

Hardcoding secrets in template parameters

Passwords and keys in parameter files end up in deployment history. Use `secureString` type for sensitive parameters, or reference Key Vault secrets directly.

Confusing parameters with variables

Parameters are supplied at deploy time (external input). Variables are computed inside the template (internal reuse). Don't put deploy-time values in variables.

Forgetting apiVersion changes behavior

Each resource type has its own API version — properties available in a newer version may not exist in an older one. Pin to a stable version and test upgrades explicitly.
